SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA1003	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	ALIGETI AKSHAYA	**Reg. No.:**	192425170
Date:	17-08-2026	**Duration:**	60 minutes

Code of Conduct

I _A.Akshaya(192425170)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____Akshaya_____

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

ACADEMIC PROJECT, ALLOCATION MONITORING AND EVALUATION MANAGEMENT SYSTEM

Assignment Questions

1. **Problem Analysis**
   - Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.

Problem Statement

The **Academic Project, Allocation Monitoring and Evaluation Management System** is proposed to improve the way academic projects are allocated, monitored, and evaluated. In the existing approach, students may be assigned projects mainly based on academic performance without giving enough importance to their individual interests and technical skills. Every student has different abilities and interests, so project allocation should consider these factors. The proposed system provides a common platform where students can enter their interests and skills, and project coordinators can allocate suitable projects. Faculty members can monitor student progress and provide guidance, while HODs and management can monitor project activities and evaluation results.

Project Objectives

- Allocate projects based on students' interests and technical skills.
- Provide a centralized platform for project management.
- Allow students to maintain their project profiles.
- Help project coordinators assign suitable projects.
- Allow faculty members to monitor project progress.
- Maintain project deadlines and progress reports.

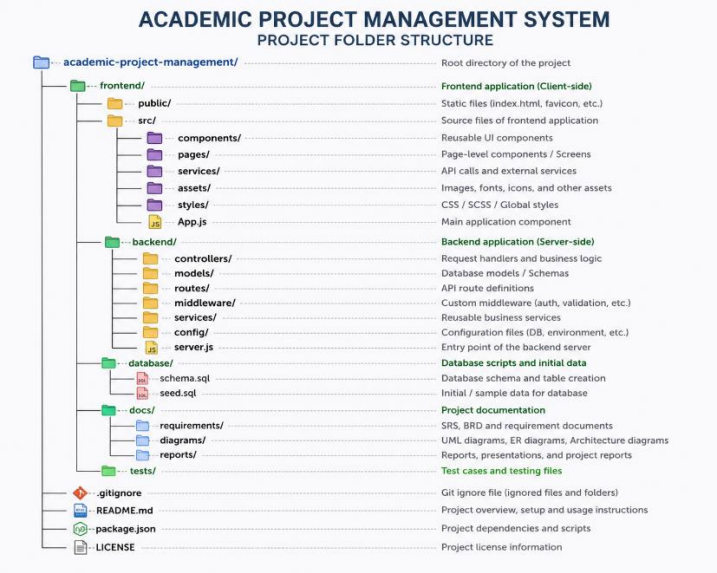
Functional Requirements

- Student, Faculty, HOD, Coordinator and Admin login.
- Student profile management.
- Interest and technical skill selection.
- Project creation and management.
- Interest-based project allocation.
- Faculty guide assignment.
- Project progress submission.
- Faculty feedback.
- Deadline management.
- Internal and external evaluation.
- Marks and feedback management.
- Dashboards for different users.
- Report generation.
- Search and filtering.
- Role-based access control.

Expected Outcomes

The expected outcome is a **centralized and user-friendly system** that makes project allocation, monitoring and evaluation easier. Students can get projects that match their interests and skills, while faculty can monitor progress efficiently. Project coordinators and HODs can track project status and deadlines. The system also maintains evaluation records digitally and improves transparency throughout the project lifecycle.

2. **Project Structure Design**
    - Design and explain the folder structure of your project repository.



    - Justify the purpose of each major folder and file included in the repository.

frontend/

Contains everything related to the user interface.

- **components/** – Reusable elements such as Navbar, LoginForm, NoticeCard and Dashboard.
- **pages/** – Complete screens such as Student Dashboard, Faculty Dashboard and Admin Dashboard.
- **services/** – API communication between frontend and backend.
- **assets/** – Images, icons and other frontend resources.
- **styles/** – CSS files and styling.
- **App.js** – Main frontend application file.

backend/

Contains the server-side application.

- **controllers/** – Handles application logic.
- **models/** – Defines database-related models.
- **routes/** – Defines API routes.
- **middleware/** – Authentication, authorization and validation.

- **services/** – Business services such as project allocation and notifications.
- **config/** – Database and server configuration.
- **server.js** – Starts the backend server.

database/

Stores database-related files.

- **schema.sql** – Creates tables such as Students, Projects, Allocations and Evaluations.
- **seed.sql** – Provides initial/sample data.

docs/

Contains project documentation such as:

- SRS
- UML diagrams
- Architecture diagrams
- Project reports

tests/

Contains test cases and testing-related files.

.gitignore

Specifies files that Git should not track, such as:

node_modules/

.env

This prevents unnecessary or sensitive files from being uploaded.

README.md

Provides information about the project, installation steps, technologies used and how to run the application.

package.json

Contains project dependencies, scripts and project information.

3. **Git Repository Initialization**
   - Create a Git repository for the project.

Step 1: Create the project folder

mkdir academic-project-management

cd academic-project-management

Step 2: Initialize Git

git init

Step 3: Create .gitignore

Example:

node_modules/

.env

*.log

Step 4: Add project files

git add .

Step 5: Create the first commit

git commit -m "Initial project structure"

Step 6: Connect to GitHub

After creating an empty repository on GitHub:

git remote add origin <repository-url>

Step 7: Push the project

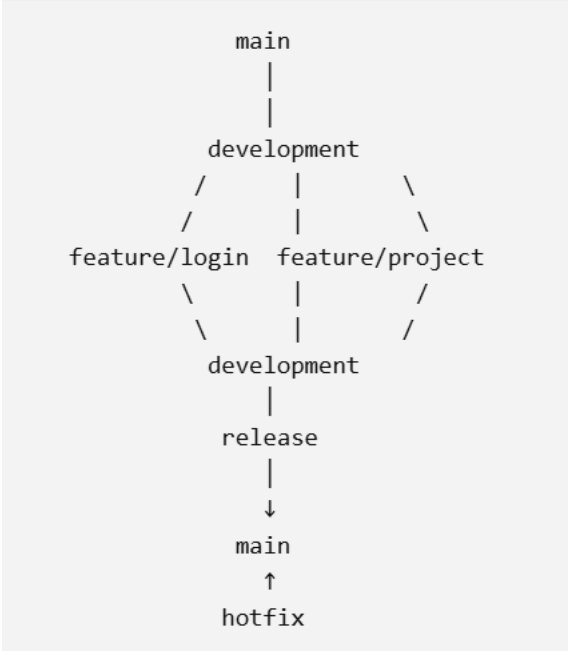
git branch -M main

git push -u origin main

- Explain the steps involved in repository initialization and describe the purpose of the essential Git files.

File	Purpose
.gitignore	Prevents unwanted/sensitive files from being tracked
.git/	Stores Git's version-control information
README.md	Explains the project and setup instructions
package.json	Stores dependencies and project scripts
.env	Stores configuration/secrets locally and should normally be ignored

4. **Git Branching Strategy**
   - Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).



   - Justify why the selected branching model is appropriate for the assigned project.

Main Branch

The main branch contains the **stable and tested version** of the project.

Only properly tested versions should be merged into main.

Development Branch

The development branch contains the latest integrated development work.

Feature branches are merged into this branch after development and testing.

Feature Branches

Separate branches can be created for individual features:

feature/login

feature/student-profile

feature/project-allocation

feature/faculty-dashboard

feature/evaluation

feature/reports

This allows different team members to work independently without disturbing the main project.

Release Branch

When a major version is ready:

release/v1.0

can be created for final testing and bug fixing before merging into main.

Hotfix Branch

If an important problem is discovered in the live/stable version:

hotfix/login-error

can be created to fix the issue quickly.

Why This Branching Strategy Is Suitable

This strategy is suitable because the project contains several modules that can be developed independently. For example, one team member can work on login while another works on project allocation.

It provides:

- Better teamwork.
- Independent feature development.
- Reduced risk of breaking the stable version.
- Easy tracking of changes.
- Organized testing.
- Easier bug fixing.
- Controlled releases.

5. **Version Control Workflow**
    - Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.

Step 1: Start from Development

```
git checkout development
```

The developer starts working from the latest development version.

Step 2: Create a Feature Branch

For example, to develop student login:

```
git checkout -b feature/student-login
```

This creates a separate branch for the login feature.

Step 3: Develop the Feature

The developer creates:

```
Login Page
Login API
Authentication
Role Selection
```

Step 4: Check Changes

```
git status
```

This shows which files have been modified.

Step 5: Stage Changes

```
git add .
```

Step 6: Commit Changes

```
git commit -m "Add student login functionality"
```

A meaningful commit message explains what was changed.

Step 7: Push the Feature Branch

```
git push -u origin feature/student-login
```

This uploads the feature branch to GitHub.

Step 8: Merge Feature into Development

After testing:

```
git checkout development
git merge feature/student-login
```

The completed login feature is now part of the development version.

Step 9: Create More Features

For example:

```
git checkout -b feature/project-allocation
```

After completing the project allocation module:

```
git add .
git commit -m "Implement interest based project allocation"
git push -u origin feature/project-allocation
```

Then merge it into development.

Conflict Resolution

Sometimes two developers may modify the same part of a file. Git may show a **merge conflict**.

For example:

```
<<<<<<< HEAD
Current development code
=======
New feature code
>>>>>>> feature/project-allocation
```

The developer should manually decide which code should be retained, remove the conflict markers, and then run:

```
git add .
git commit -m "Resolve merge conflict"
```

The purpose of conflict resolution is to combine changes correctly without losing important work.

Repository Synchronization

Before starting new work, developers should get the latest changes:

```
git pull origin development
```

After completing their work:

```
git push origin feature/student-login
```

- Explain the purpose of each Git operation performed.

  Create Repository

  ↓

  Initialize Git

  ↓

  Create main + development

  ↓

  Create Feature Branch

  ↓

  Develop Feature

  ↓

  git add

↓

git commit

↓

git push

↓

Code Review / Testing

↓

Merge into Development

↓

Integration Testing

↓

Create Release Branch

↓

Final Testing

↓

Merge into Main

↓

Stable Project Version

6. **Commit History Analysis**
   - Present the commit history of the project.

     Commit 1: Initial project structure

     Commit 2: Add user registration and login

     Commit 3: Add student profile management

     Commit 4: Add student interest and skill selection

     Commit 5: Implement project allocation module

Commit 6: Add faculty monitoring dashboard

Commit 7: Add project progress tracking

Commit 8: Add evaluation and feedback module

Commit 9: Fix project allocation validation

Commit 10: Add report generation

Commit 11: Improve role-based access control

Commit 12: Prepare version 1.0 release

- Explain how commit messages follow good version control practices and contribute to project maintenance.

  - Commit messages should be **short and meaningful**.
  - Each commit should describe one major change.
  - Avoid messages such as `"update"` or `"changes done"`.
  - Use messages such as `"Add student profile module"` or `"Fix login validation"`.
  - Regular commits make it easier to find and correct errors.
  - Commit history also helps new developers understand the development process.

7. **Docker Environment Design**
   - Explain why Docker is suitable for the assigned project.

Docker is suitable for this project because the system contains multiple components such as the frontend, backend and database. Docker allows these components to run in separate, controlled environments.

For example, the project can contain:

Frontend Container

↓

Backend Container

↓

Database Container

- Identify the application components that require containerization.

- **Frontend** – Contains the user interface of the system.
- **Backend** – Handles APIs, authentication, project allocation and business logic.
- **Database** – Stores students, projects, allocations, evaluations and feedback.
- **Optional Nginx container** – Can be used to serve the frontend and manage requests.

Benefits of Docker

- Same environment for all developers.
- Reduces installation problems.
- Easy application deployment.
- Separates frontend, backend and database.
- Makes testing easier.
- Supports future scaling.

8. **Dockerfile Development**
   - Design and explain the Dockerfile required to containerize the application.

backend/

└── Dockerfile

Example:

```
FROM node:20


WORKDIR /app


COPY package*.json ./


RUN npm install
```

```
COPY . .

EXPOSE 5000

CMD ["npm", "start"]
```

- Describe the purpose of each instruction used in the Dockerfile.

Instruction	Purpose
FROM node:20	Provides the Node.js environment
WORKDIR /app	Sets the working directory inside the container
COPY package*.json ./	Copies dependency files
RUN npm install	Installs required packages
COPY . .	Copies backend source code
EXPOSE 5000	Indicates the backend port
CMD ["npm", "start"]	Starts the backend application

9. **Docker Compose Design**
   - Develop a Docker Compose configuration for the project.

```
academic-project-management/
│
├── frontend/
│   └── Dockerfile
│
├── backend/
│   └── Dockerfile
│
├── database/
│   └── schema.sql
```

```
│
└── docker-compose.yml
```

docker-compose.yml

For a Node.js full-stack application using MySQL, you can use:

```
services:


  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    depends_on:
      - backend


  backend:
    build: ./backend
    ports:
      - "5000:5000"
    environment:
      DB_HOST: database
      DB_USER: root
      DB_PASSWORD: root
      DB_NAME: academic_project
    depends_on:
      - database
```

```
database:
  image: mysql:8
  environment:
    MYSQL_ROOT_PASSWORD: root
    MYSQL_DATABASE: academic_project
  ports:
    - "3306:3306"
  volumes:
    - db_data:/var/lib/mysql
    - ./database/schema.sql:/docker-entrypoint-initdb.d/schema.sql


volumes:
  db_data:
```

- Explain the services, networking, volumes, environment variables, and dependencies used.

Frontend

frontend:

Runs the user interface of the Academic Project Management System.

Backend

backend:

Runs the server-side application and provides APIs for login, project allocation, monitoring and evaluation.

Database

database:

Runs MySQL and stores the application's data.

depends_on

depends_on:

Ensures that the required services are started before the dependent service.

Volume

db_data:

Keeps database data persistent even when the database container is stopped or recreated.

Running the Complete Application

After creating the Dockerfiles and docker-compose.yml, run:

docker compose up --build

To stop the application:

docker compose down

10. **Container Deployment and Testing**
    - Demonstrate the execution of the application using Docker containers.

Deployment Steps

1. Open the project folder in the terminal.
2. Make sure the Dockerfile files and docker-compose.yml are available.
3. Build and start the containers using:

   docker compose up --build

4. Check the running containers using:

   docker compose ps

5. Open the frontend in the browser using:

   http://localhost:3000

6. The backend API can be tested using:

   http://localhost:5000

   - Explain the testing procedure and provide evidence that the application runs successfully.

Testing Procedure

The following features can be tested:

- Check whether the login page opens correctly.
- Test student registration and login.
- Check student profile creation.
- Test interest and skill selection.
- Check project allocation.
- Test faculty project monitoring.
- Submit and check project progress.
- Test evaluation and feedback.
- Check database records.
- Verify communication between frontend, backend and database.

Evidence of Successful Execution

The successful execution can be demonstrated using screenshots of:

- Docker Desktop showing running containers.
- Terminal showing docker compose up successfully.
- docker compose ps showing frontend, backend and database containers.
- Login page running in the browser.
- Student dashboard.
- Project allocation page.
- Faculty monitoring dashboard.

11. **Benefits of Git and Docker**
    - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.

Git Benefits

1. Collaboration:

Different team members can work on different features such as login, project allocation and evaluation using separate branches.

2. Version Management:

Git stores the history of project changes, allowing developers to check previous versions and restore code when required.

3. Easy Error Recovery:

If a new change causes an error, the team can identify the commit that introduced the problem and correct it.

4. Organized Development:

Feature, development, release and hotfix branches keep the project development organized.

Docker Benefits

5. Portability:

The application can run on different computers as long as Docker is available.

6. Easy Deployment:

Frontend, backend and database can be started together using Docker Compose.

7. Consistent Environment:

All developers can use the same Node.js, database and application environment.

8. Scalability:

Individual services can be scaled when the number of students or users increases.

9. Maintainability:

Separating the application into containers makes it easier to update or replace individual components.

12. **Challenges and Improvements**
    - Identify the challenges encountered during implementation.

Challenges Encountered

- Understanding Git commands and branching can be difficult for beginners.
- Merge conflicts may occur when multiple developers modify the same files.
- Maintaining meaningful commit messages requires discipline.
- Docker installation and configuration may create initial difficulties.
- Connecting frontend, backend and database containers can cause configuration problems.
- Database connection errors may occur between the backend and MySQL container.
- Port conflicts may occur when another application is using the same port.
- Managing environment variables and database passwords requires care.
- Debugging errors inside containers can be more difficult than debugging locally.

    - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Possible Improvements

Git Workflow Improvements

- Use a fixed branch naming convention such as feature/login.
- Follow meaningful commit messages.
- Perform code reviews before merging.
- Protect the main branch from direct changes.
- Use pull requests for feature integration.
- Add automated testing before merging code.

Docker Improvements

- Use separate development and production configurations.
- Store passwords and database credentials using environment variables or secrets.
- Add health checks to verify that services are running correctly.
- Use smaller production Docker images to improve performance.
- Add automated Docker builds through CI/CD.
- Use Docker volumes for persistent database storage.

Future Enhancements

In the future, the project can be improved by adding:

- Automated GitHub Actions CI/CD.
- Automatic Docker image building.
- Cloud deployment.
- Online project presentation scheduling.
- Email and mobile notifications.
- AI-based project recommendations based on student skills.
- Advanced project progress analytics.
- Automatic report generation.
- Backup and recovery mechanisms.
- Monitoring and logging for production deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow &	Demonstrates meaningful commits, branching, merging, synchronization,	Most Git operations demonstrated correctly	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.

Commit History(10%)	conflict resolution, and professional commit messages with clear history.	with good commit history.		
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25